Claims Management Application

QA findings: bugs found, evidence and fixes

Prepared by	Chan Basha
For	Chubb APAC QA Engineer take-home, hiring panel walkthrough
Date	12 October 2026
Repository	github.com/chansgrowthhub-arch/chubb-claimns-qa-assessment
Baseline	Commit `1dea234`: the application exactly as received

How to read this report. Every screenshot is a real artefact. The terminal images are captured from actual test runs, the code images are read directly from the source files at the line numbers shown, and the diffs come from `git diff`. "Before" runs use the untouched baseline commit. "After" runs use the current code.

Scope limit. The machine used had no Docker, so the full stack (Kafka, Keycloak, Postgres) was never started. Unit, component and contract tests were executed. The Playwright end-to-end suite is written and compiles, but has not been run.

1. Summary

11	3	2	734
findings, including 4 high severity	fixed in this work	failing defect tests that reproduce open bugs	tests passing after the work (claims 237, BFF 219, UI 278)

#	Finding	Severity	Status	Proof
1	Real-time updates are silently off in the default stack	High	Open	Config and code review
2	CDC message field mismatch: claim id arrives empty	High	Open	Failing defect test
3	Logout rejected by CSRF protection, so the session survives	High	Open	Contract test, E2E test marked as expected failure
4	Logout fails when the access token has expired	High	Open	Failing defect test
5	The shipped test suite could not pass	Medium	Fixed	Before and after runs
6	Shared bearer token across concurrent requests	Medium	Open	Code review
7	`changedBy` records the claim owner instead of the admin	Medium	Open	Code review
8	Theme preference lost in every new tab	Low	Fixed	Shipped tests now pass
9	E2E runner broken: setup file missing	Low	Fixed	Playwright lists 10 specs
10	Status-change error message discarded by the BFF	Low	Open	Code review
11	Smaller contract and security smells	Low	Open	Code review

What "Fixed" and "Open" mean here

Fixed means the change is committed and verified by a passing test run. Two fixes were to the test suite and one was a one-line application change.

Open means the bug is still in the application. As a QA engineer I proved and pinned these bugs rather than silently changing product behaviour. Each open bug has a recommended fix in this report. Bugs 2 and 4 also have a failing test that turns green once the fix lands.

Risk ranking that drove the work

- **Claim business rules.** Amounts, dates and the status state machine carry direct financial impact.
- **Kafka to WebSocket pipeline.** Three hand-written JSON contracts with no schema enforcement. Failures are silent.
- **Role-based access and claim ownership.** A gap here leaks other customers' claims.
- **Authentication and session lifecycle.** The login, refresh and logout code is hand-rolled.

2. Bug 1: Real-time updates are silently off in the default stack

High **Open**

What happens	A claimant submits a claim or an admin changes a status, and nobody receives a live update. No toast appears and the claims list does not refresh until a manual reload. Nothing errors.
Why	The compose file turns CDC mode on. In CDC mode the claims service publishes no Kafka events and relies on Debezium. Debezium is a separate, optional stack that the quick-start never starts.
Impact	The headline real-time feature does not work for anyone following the README.

docker-compose.apps.yml

```
16      environment:
17        SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/demo_app
18        SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:29092
19        SPRING_DATA_REDIS_HOST: redis
20        SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_ISSUER_URI: http://localhost:8180/realms/demo-app
21        SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_JWK_SET_URI: http://keycloak:8080/realms/demo-app/protocol/openid-connect/certs
22        # Set to true when running docker-compose.cdc.yml alongside this stack
23        APP_EVENTS_CDC_ENABLED: "true"
24      networks:
```

Compose sets **CDC mode ON** for claims-service ...

Figure 1.1: The compose file sets CDC on for the claims service.

README.md (as received, commit 1dea234)

```
84  ### Mode 1: Direct Kafka (default)
85
86  The Claims Service publishes events directly to Kafka after each write. No extra setup needed.
87
88  `APP_EVENTS_CDC_ENABLED=false` is set by default in `docker-compose.apps.yml`. Just run the normal startup above.
```

... while the README promises direct-Kafka mode (CDC **OFF**) is the default.

Figure 1.2: The README states the opposite default.

code/claims-service/src/main/java/com/example/demo/application/usecases/CreateClaimUseCase.java

```
47          Claim savedClaim = claimRepository.save(claim);
48
49          if (cdcEnabled) {
50              LOG.info("CDC enabled — event will be published by Debezium for claimId={}", savedClaim.getClaimId());
51          } else {
52              LOG.info("CDC disabled — publishing {} event(s) directly for claimId={}",
53                      claim.getDomainEvents().size(), savedClaim.getClaimId());
54              claim.getDomainEvents().forEach(eventPublisher::publishEvent);
55              claim.clearEvents();
56          }
```

With cdcEnabled=true the service only logs - it publishes nothing and relies on Debezium, which the quick-start never starts.

Figure 1.3: With CDC on, the use case only writes a log line.

Recommended fix (not applied)

```
docker-compose.apps.yml
-       APP_EVENTS_CDC_ENABLED: "true"
+       APP_EVENTS_CDC_ENABLED: "false"   # direct Kafka by default; start-cdc.sh docs say flip to true
```

Test coverage. `CreateClaimUseCaseTest` now asserts that with CDC off the claim-submitted event is published and then cleared. The end-to-end confirmation needs the live stack.

3. Bug 2: CDC message field mismatch, so the claim id arrives empty

High Open

What happens	A change event that follows the documented contract reaches the browser with an empty claim id, so the UI cannot match it to any row.
Why	Three parts of the system disagree on one field name. The BFF reads `claim_id`. The committed AsyncAPI schema and the test-only event-injection endpoint both use `id`.
Why it slipped	The shipped consumer tests only checked that a broadcast happened. They never checked what was in it.

code/bff-service/src/main/java/com/example/demo/adapter/in/messaging/ClaimEventKafkaConsumer.java

```
114     private void handleCdcUpdate(JsonNode cdcEnvelope) {
115         JsonNode before = cdcEnvelope.path("before");
116         JsonNode after = cdcEnvelope.path("after");
117
118         String claimId = after.path("claim_id").asText();
119         String newStatus = after.path("status").asText();
120         String oldStatus = before.path("status").asText();
121         String userId = after.path("user_id").asText(null);
```

BFF consumer reads the claim id from **after.claim_id**.

Figure 2.1: The consumer expects `claim_id`.

code/claims-service/src/main/resources/asyncapi/schemas/DebeziumClaimsMessage.json

```
74           "type": "object",
75           "description": "Shared claim fields reused in CDC after schema and domain events",
76           "properties": {
77             "id": {
78               "type": "string",
79               "format": "uuid"
80             },
81             "user_id": {
```

The committed message contract names the same field **id**.

Figure 2.2: The schema documents `id`.

code/claims-service/src/main/java/com/example/demo/adapter/in/web/TestEventController.java

```
62         return """
63             {
64               "op": "u",
65               "after": {
66                 "id": "%s",
67                 "status": "%s",
68                 "updated_at": %d
69               },
```

The E2E event-injection endpoint also emits **id** - so every injected event reaches the UI with an empty claimId.

Figure 2.3: The injection endpoint emits `id`.

3. Bug 2 continued: proof and fix

bff-service - known-defect tests (tag "defect"), FAIL BY DESIGN

```
$ mvn test -Dgroups=defect -Dsurefire.excludedGroups=integration
[ERROR]   ClaimEventKafkaConsumerContentTest.should_BroadcastClaimId_When_CdcPayloadMatchesCommittedAsyncApiSchema:148 [claimId must survive
a schema-conformant CDC payload (currently arrives empty)]
expected: "d4e5f6a7-b8c9-0123-4567-89abcdef0123"
 but was: ""
[ERROR]   AuthCsrfContractTest.should_ClearCookies_When_LogoutCalledWithExpiredAccessToken:96 Status expected:<204> but was:<401>
[ERROR] Tests run: 2, Failures: 2, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
```

Figure 2.4: The new defect-tagged tests fail by design. The first shows `expected "d4e5f6a7..." but was ""`. The second is Bug 4.

Recommended fix (not applied)

Real Debezium uses the actual Postgres column name, `claim_id`. Align the schema and the injection endpoint to it, and enable the existing schema validator in CI.

```
TestEventController.buildMockCDCPayload:   "id": "%s"   ->   "claim_id": "%s"
DebeziumClaimsMessage.json (before/after): "id"          ->   "claim_id"
```

How the defect tests are kept honest

git diff 1dea234..HEAD -- code/bff-service/pom.xml

```
@@ -45,5 +45,5 @@
                <skipAsyncApiGeneration>true</skipAsyncApiGeneration>
                <pact.version>4.6.3</pact.version>
-               <surefire.excludedGroups>integration</surefire.excludedGroups>
+               <surefire.excludedGroups>integration,defect</surefire.excludedGroups>
        </properties>
        <dependencies>
```

Known-defect tests are tagged "defect" and kept out of the default run, so CI stays green and the bugs stay reproducible.

Figure 2.5: Defect tests are tagged "defect" and kept out of the default run. CI stays green, and each bug stays reproducible with one command.

4. Bug 3: Logout is rejected by CSRF protection

High **Open**

What happens	The user clicks Logout and sees the login page. The server session and the httpOnly `access_token` cookie survive. Navigating back to a protected page still works.
Why	Logout is not on the BFF's CSRF exemption list, so the BFF requires an `X-XSRF-TOKEN` header. The UI sends a plain fetch without one. The BFF answers 403, and the UI clears its local state anyway.
Why it slipped	The shipped controller test always added a CSRF token, so it never exercised the request the browser actually sends.

code/bff-service/src/main/java/com/example/demo/config/SecurityConfig.java

```
55                  .csrf(csrf -> csrf
56                          .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
57                          .csrfTokenRequestHandler(csrfTokenRequestHandler)
58                          .ignoringRequestMatchers(
59                                  "/api/auth/signup",
60                                  "/api/auth/login",
61                                  "/api/auth/refresh",
62                                  "/api/ws/**",
63                                  "/actuator/**"
64                          ))
```

CSRF exemptions: signup, login, refresh ... **logout is not listed**, so it requires an X-XSRF-TOKEN header.

Figure 3.1: The CSRF exemptions cover signup, login and refresh, but not logout.

code/demo-app-ui/src/components/layout/app-header.tsx

```
16  const handleLogout = async () => {
17    setIsLoggingOut(true);
18    try {
19      await fetch('/api/auth/logout', {
20        method: 'POST',
21        credentials: 'include',
22      });
23    } catch (error) {
24      console.error('Logout error:', error);
25    }
26
27    logout();
28    router.push('/login');
29  };
```

UI logout: raw fetch with **no CSRF header** -> BFF answers 403 -> the catch never fires (403 is not an exception) -> local state is cleared anyway.

Figure 3.2: The UI logout sends no CSRF header and ignores the 403.

Test coverage. `AuthCsrfContractTest` sends the exact request the browser makes and proves it gets a 403, with Keycloak invalidation never called. The Playwright test `logout invalidates the server session` is marked as an expected failure until this is fixed.

Recommended fix (not applied)

```
app-header.tsx: send the CSRF header like the generated API client already does
  const csrf = document.cookie.match(/XSRF-TOKEN=([^;]+)/)?.[1];
  await fetch('/api/auth/logout', { method: 'POST', credentials: 'include',
    headers: csrf ? { 'X-XSRF-TOKEN': decodeURIComponent(csrf) } : {} });
  ...and only clear local state once the response is ok, or show an error.
```

5. Bug 4: Logout fails when the access token has expired

High Open

What happens	A user whose 5-minute access cookie is stale cannot log out. The BFF answers 401 before the logout code runs, so the 30-minute refresh cookie is never cleared.
Why	A filter converts any `access_token` cookie into a Bearer header. Spring then rejects the expired token before the controller is reached.
How found	While writing the passing CSRF test, my first version included a cookie and got 401 instead of 204. I kept the case as a defect test rather than deleting the cookie to make the test pass.

code/bff-service/src/main/java/com/example/demo/config/CookieAuthenticationFilter.java

```
36          // Extract access_token from cookies
37          String accessToken = extractAccessTokenFromCookies(request);
38
39          if (accessToken != null) {
40              // Wrap request to add Authorization header
41              HttpServletRequest wrappedRequest = new HttpServletRequestWrapper(request) {
42                  @Override
43                  public String getHeader(String name) {
44                      if (AUTHORIZATION_HEADER.equalsIgnoreCase(name)) {
45                          return BEARER_PREFIX + accessToken;
46                      }
47                      return super.getHeader(name);
48                  }
49              };
50              filterChain.doFilter(wrappedRequest, response);
```

Any access_token cookie becomes a Bearer header. An **expired** token is rejected with 401 before logout ever runs.

Figure 4.1: The filter turns every access_token cookie into an Authorization header.

Figure 2.4 in the Bug 2 section shows the failing proof: `AuthCsrfContractTest.should_ClearCookies_When_LogoutCalledWithExpiredAccessToken`, expected 204 but got 401.

Recommended fix (not applied)

```
SecurityConfig: .requestMatchers("/api/auth/logout").permitAll()
CookieAuthenticationFilter: skip the cookie-to-bearer conversion for /api/auth/logout
AuthController.logout: always clear both cookies; invalidate the Keycloak session best-effort
```

6. Bug 5: The shipped test suite could not pass

Medium **Fixed**

The brief mentioned one planted broken test. Running the suites showed the problem was wider. No CI pipeline could have been green, so the configured 90% coverage and 80% mutation thresholds were never actually enforced.

5a. The planted compile error

claims-service - as received (baseline commit 1dea234)

```
$ mvn test
[ERROR] COMPILATION ERROR :
[INFO] -------------------------------------------------------------
[ERROR] code/claims-service/src/test/java/com/example/demo/application/usecases/UpdateClaimStatusUseCaseTest.java:[45,19] constructor
UpdateClaimStatusUseCase in class com.example.demo.application.usecases.UpdateClaimStatusUseCase cannot be applied to given types;
  required:
com.example.demo.domain.claim.ClaimRepository,com.example.demo.user.domain.UserRepository,io.micrometer.core.instrument.MeterRegistry,org.sp
ringframework.context.ApplicationEventPublisher
  found:
com.example.demo.domain.claim.ClaimRepository,com.example.demo.user.domain.UserRepository,io.micrometer.core.instrument.MeterRegistry
  reason: actual and formal argument lists differ in length
[INFO] -------------------------------------------------------------
[INFO] BUILD FAILURE
```

Figure 5.1: The baseline does not compile. The test is missing the new `ApplicationEventPublisher` argument.

git diff 1dea234..HEAD -- code/claims-service/src/test/java/com/example/demo/application/usecases/UpdateClaimStatusUseCaseTest.java

```
1  @@ -14,4 +14,5 @@ import com.example.demo.user.domain.UserRole;
2   import io.micrometer.core.instrument.MeterRegistry;
3   import io.micrometer.core.instrument.simple.SimpleMeterRegistry;
4  +import org.springframework.context.ApplicationEventPublisher;
5   import org.junit.jupiter.api.BeforeEach;
6   import org.junit.jupiter.api.Test;
7  @@ -32,4 +33,5 @@ class UpdateClaimStatusUseCaseTest {
8       private UserRepository userRepository;
9       private MeterRegistry meterRegistry;
10 +     private ApplicationEventPublisher eventPublisher;
11      private UpdateClaimStatusUseCase useCase;
12
13 @@ -43,5 +45,6 @@ class UpdateClaimStatusUseCaseTest {
14          userRepository = mock(UserRepository.class);
15          meterRegistry = new SimpleMeterRegistry();
16 -        useCase = new UpdateClaimStatusUseCase(claimRepository, userRepository, meterRegistry);
17 +        eventPublisher = mock(ApplicationEventPublisher.class);
18 +        useCase = new UpdateClaimStatusUseCase(claimRepository, userRepository, meterRegistry, eventPublisher);
19      }
20
```

Fix applied: supply the missing ApplicationEventPublisher dependency.

Figure 5.2: Fix applied.

5b. Two more stale test classes appeared once it compiled

claims-service - after fixing ONLY the planted compile error

```
$ mvn test
[ERROR] Failures / Errors:
[ERROR]   KafkaDomainEventPublisherTest.should_PropagateValidationException_When_ValidatorRejectsInvalidClaimStatus:214
[ERROR]   KafkaDomainEventPublisherTest.should_PropagateValidationException_When_ValidatorRejectsMissingRequiredField:191
[ERROR]   KafkaDomainEventPublisherTest.should_ThrowIllegalStateException_When_SerializationFails:141
[ERROR]   KafkaDomainEventPublisherTest.should_CallValidatorWithCorrectArgs_When_ClaimSubmittedEventPublished:167 » NullPointer Cannot
invoke "com.fas ...
[ERROR]   KafkaDomainEventPublisherTest.should_PublishEventToKafka_When_ClaimStatusChangedEventProvided:90 » NullPointer Cannot invoke
"com.fasterxml. ...
[ERROR]   KafkaDomainEventPublisherTest.should_PublishEventToKafka_When_ClaimSubmittedEventProvided:67 » NullPointer Cannot invoke
"com.fasterxml.jack ...
[ERROR]   CreateClaimUseCaseTest.should_CreateClaim_When_ValidCommandProvided:66 » NullPointer
[ERROR]   CreateClaimUseCaseTest.should_PublishClaimSubmittedEvent_When_ClaimCreated:109 » NullPointer
[ERROR]   CreateClaimUseCaseTest.should_SetStatusToSubmitted_When_ClaimCreated:129 » NullPointer
[ERROR] Tests run: 238, Failures: 3, Errors: 6, Skipped: 0
[INFO] BUILD FAILURE
```

Figure 5.3: Nine failures after fixing only the planted error.

code/claims-service/src/test/java/com/example/demo/adapter/out/messaging/KafkaDomainEventPublisherTest.java (as received, commit 1dea234)

```
62          when(objectMapper.writeValueAsString(event)).thenReturn("{\"eventId\":\"test\"}");
63          doNothing().when(schemaValidator).validate(anyString(), anyString());
64          when(kafkaTemplate.send(anyString(), anyString(), anyString()))
65                  .thenReturn(CompletableFuture.completedFuture(null));
66
67          publisher.publish(event);
68
69          verify(kafkaTemplate).send(eq("claim-events"), eq(claimId.toString()), eq("{\"eventId\":\"test\"}"));
70          verify(schemaValidator).validate("{\"eventId\":\"test\"}", "ClaimSubmittedMessage");
```

Shipped test mocks writeValueAsString(event) - production now builds the envelope with createObjectNode(), so the mock returns null.

Figure 5.4: Root cause in the publisher test. It mocked a method the production code no longer calls.

Fix applied. I added the missing mock to `CreateClaimUseCaseTest`, which now asserts that the event is published and cleared. I rewrote `KafkaDomainEventPublisherTest` with a real JSON mapper so it asserts the actual Kafka envelope the BFF depends on.

5c. Eight UI tests were failing as received

demo-app-ui - as received (baseline commit 1dea234)

```
$ npm test   (vitest run)
 FAIL  tests/unit/use-theme-store.test.ts > useThemeStore > persists theme to localStorage when changed AssertionError: expected null to be
truthy
 FAIL  tests/unit/use-theme-store.test.ts > useThemeStore > persists isDark to localStorage when toggled AssertionError: expected null to be
truthy
 FAIL  src/features/claims/components/wizard-step-2.test.tsx > WizardStep2 > Character Counter > should show primary color when approaching
maximum
     Error: Test timed out in 5000ms.
 FAIL  src/features/claims/components/wizard-step-2.test.tsx > WizardStep2 > Character Counter > should show muted color when character
count is valid
     TestingLibraryElementError: Unable to find an element with the text: 50/1000. This could be because the text is broken
 FAIL  src/features/claims/components/wizard-step-2.test.tsx > WizardStep2 > Field Validation > should show error for description shorter
than 10 characters
     TestingLibraryElementError: Unable to find an element with the text: /Description must be at least 10 characters/i.
 FAIL  src/features/claims/components/wizard-step-2.test.tsx > WizardStep2 > Navigation > should call updateFormData and nextStep when form
is valid and Next is clicked
     AssertionError: expected "vi.fn()" to be called with arguments: [ Array(1) ]
 FAIL  src/features/claims/components/wizard-step-2.test.tsx > WizardStep2 > Accessibility > should have proper aria-invalid when error
exists
     Error: expect(element).toHaveAttribute("aria-invalid", "true") // element.getAttribute("aria-invalid") === "true"
 FAIL  src/features/claims/components/wizard-step-2.test.tsx > WizardStep2 > Accessibility > should have aria-describedby linking to error
and counter
     AssertionError: expected 'description-counter' to contain 'description-error'

 Test Files  2 failed | 23 passed (25)
      Tests  8 failed | 258 passed (266)
```

Figure 5.5: Baseline UI run. Two tests failed in the theme store and six in the claim wizard.

Root cause. One wizard test typed 950 characters one keystroke at a time. That took longer than the 5-second timeout, and the leftover keystrokes leaked into the next five tests. I did not raise the timeout. I changed the test to paste the text, which is still a real user action.

git diff 1dea234..HEAD -- code/demo-app-ui/src/features/claims/components/wizard-step-2.test.tsx

```
1   @@ -111,6 +111,9 @@ describe('WizardStep2', () => {
2          const textarea = screen.getByLabelText(/Describe what happened/i);
3
4   +      // user.type() of 950 chars exceeds the 5s test timeout and its leaked keystrokes
5   +      // poison every later test in this file; paste is equally user-driven and instant.
6          const longText = 'a'.repeat(950);
7   -      await user.type(textarea, longText);
8   +      await user.click(textarea);
9   +      await user.paste(longText);
10
11         await waitFor(() => {
```

Fix applied: paste instead of typing 950 keystrokes (5 s timeout blew up and leaked into later tests).

Figure 5.6: Fix applied.

The two theme failures were a real application bug, covered as Bug 8.

7. Bug 6: Shared bearer token across concurrent requests

Medium Open

The BFF keeps one HTTP client per controller and overwrites its bearer token on every request. Under concurrent load, one user's call to the claims service can go out with another user's token or the service-account token. That is a correctness risk and an authorization leak. This needs a concurrency test against the live stack, so it is flagged from code review rather than proven.

code/bff-service/src/main/java/com/example/demo/adapter/in/web/AdminController.java

```
148     private void setServiceAccountBearerToken() {
149         String token = serviceAccountTokenProvider.getServiceAccountToken();
150         apiClient.setBearerToken(token);
151     }
152
153     private void setUserBearerToken() {
154         var authentication = org.springframework.security.core.context.SecurityContextHolder
155                 .getContext().getAuthentication();
156         if (authentication != null && authentication.getPrincipal() instanceof org.springframework.security.oauth2.jwt.Jwt jwt) {
157             apiClient.setBearerToken(jwt.getTokenValue());
158         } else {
159             throw new IllegalStateException("No user JWT token found in security context");
```

One shared ApiClient per controller; each request **overwrites** its bearer token. Concurrent requests can send another user's token.

Figure 6.1: The token is set on a shared client instance.

Recommended fix. Resolve the token per request, for example with an interceptor that reads the current security context, instead of mutating a shared client.

8. Bug 7: `changedBy` records the claim owner instead of the admin

Medium Open

The schema says `changedBy` is "the admin user who changed the status", but the domain fills it with the claim owner. Owner notifications only work by accident, because the BFF uses this field as the notification target. A naive fix would therefore break owner notifications. The audit trail is wrong today.

code/claims-service/src/main/java/com/example/demo/domain/claim/Claim.java

```
130         this.domainEvents.add(new ClaimStatusChanged(
131             UUID.randomUUID().toString(),
132             this.claimId,
133             oldStatus,
134             newStatus,
135             this.userId,
136             Instant.now()
137         ));
```

changedBy is filled with the claim **owner** (this.userId), not the admin who made the change.

Figure 7.1: `this.userId` is the claim owner.

Recommended fix. Carry both `ownerId` and `changedBy` in the event, and route notifications on `ownerId`. Add a contract test first.

9. Bug 8: Theme preference lost in every new tab

Low Fixed

The shipped tests specify that the theme persists in `localStorage`, but the store used `sessionStorage`. I changed the code to match the tests, because the tests describe the intended behaviour.

git diff 1dea234..HEAD -- code/demo-app-ui/src/features/theme/stores/use-theme-store.ts

```
@@ -23,5 +23,7 @@ export const useThemeStore = create<ThemeStore>()(
     {
       name: 'theme-storage',
-      storage: createJSONStorage(() => sessionStorage),
+      // Shipped tests specify localStorage (theme should survive new tabs/sessions);
+      // implementation used sessionStorage, silently resetting user preference.
+      storage: createJSONStorage(() => localStorage),
     }
   )
```

Fix applied: theme preference now persists in localStorage, as the tests specify.

Figure 8.1: One-line application fix. The two shipped theme tests now pass.

10. Bug 9: E2E runner broken because the setup file is missing

Low Fixed

code/demo-app-ui/playwright.config.ts (as received, commit 1dea234)

```
    baseURL: 'http://localhost:3001',
    trace: 'on-first-retry',
  },
  globalSetup: './tests/e2e/global-setup.ts',
  projects: [
```

Shipped config points to a global-setup file that did not exist in the repo.

Figure 9.1: The config referenced `tests/e2e/global-setup.ts`, which did not exist, so `npx playwright test` failed before any test ran.

Fix applied. I added the global setup, which logs in each seeded user once and saves the session, along with page objects and 10 specs.

11. Bug 10: Status-change error message discarded

Low Open

code/bff-service/src/main/java/com/example/demo/adapter/in/web/AdminController.java

```
            return ResponseEntity.ok(adminClaimResponseMapper.toClaimResponse(claimsResponse));
        } catch (ApiException exception) {
            if (exception.getCode() == 400) {
                return ResponseEntity.badRequest().build();
            }
```

The claims-service ProblemDetail ("Cannot transition from X to Y") is discarded and an empty 400 is returned.

Figure 10.1: The BFF returns an empty 400, so the admin sees a generic error instead of the reason.

12. Bug 11: Smaller contract and security smells

Low Open

- **Money as floating-point.** The domain uses exact decimals, but both APIs declare `number/double`, so every hop converts decimal to double and back. Storage is safe because the column is `DECIMAL(12,2)`.
- **Inconsistent minimum amount.** The domain accepts 0.001, while the API rejects anything below 0.01. A test documents this.
- **Token in the URL.** The WebSocket handshake accepts the JWT as a `?token=` query parameter, which leaks into logs and browser history.
- **Token outlives its cookie.** The access token lives 24 hours in Keycloak, but its cookie lives 5 minutes.
- **Truncation mismatch.** The UI truncates descriptions at 50 characters while the summary API truncates at 100.

13. What we tested in the UI

New component tests drive the login page and the claims table the way a user would, by label and role, with the backend mocked at the network boundary.

New component tests - login page and claims table

```
$ npx vitest run login-page.test.tsx claims-table.test.tsx --reporter=verbose
✓ ClaimsTable > renders one row per claim with formatted amount, date, and status label 204ms
✓ ClaimsTable > truncates long descriptions but keeps the full text available as a tooltip 5ms
✓ ClaimsTable > highlights only the row whose claim was just updated in real time 6ms
✓ ClaimsTable > invokes onViewDetails with the full claim id 79ms
✓ ClaimsTable > copies the full claim id to the clipboard from the copy affordance 94ms
✓ LoginPage > submits credentials to the BFF login endpoint with cookies enabled 735ms
✓ LoginPage > stores the authenticated user in the auth store on success 632ms
✓ LoginPage > shows "Invalid email or password" on a 401 without storing a user 683ms
✓ LoginPage > shows a network error message when the BFF is unreachable 660ms
✓ LoginPage > blocks submission and shows a validation message for a malformed email 703ms
✓ LoginPage > requires a password before submitting 306ms
✓ LoginPage > announces the session-expired message when redirected with session_expired=true 17ms
Test Files  2 passed (2)
Tests  12 passed (12)
```

Figure 13.1: 12 new component tests, all passing.

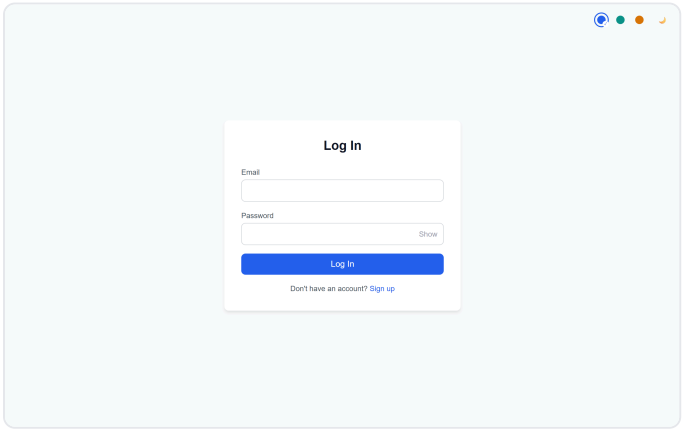


Figure 13.2: Login page, rendered locally on the dev server with no backend.

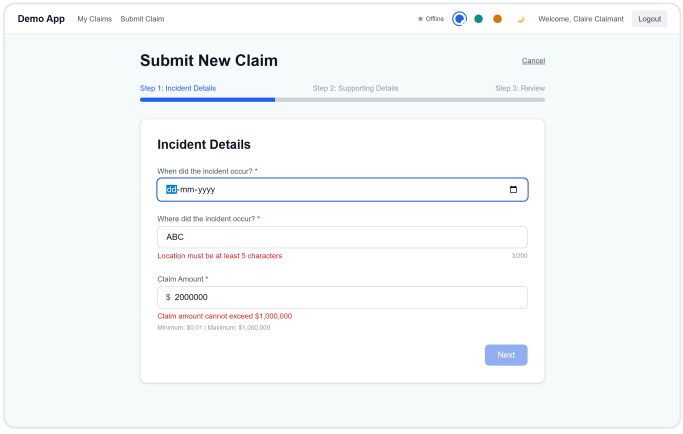


Figure 13.3: Claim wizard step 1 rejecting a 3-character location and an amount over $1,000,000. Real client-side validation; the wizard tests cover these limits.

14. Results after the work

All three suites - current HEAD

```
$ mvn test  (claims-service)  |  mvn test  (bff-service)  |  npm test  (demo-app-ui)
# claims-service
[INFO] Tests run: 237, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

# bff-service
[INFO] Tests run: 219, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

# demo-app-ui
Test Files  27 passed (27)
Tests  278 passed (278)
```

Figure 14.1: All three default suites green on the current code.

Suite	As received	After
Claims service	Did not compile; then 9 failures	237 passed
BFF	213 passed, but its tests missed bugs 2–4	219 passed, plus 2 defect tests failing by design
UI	8 failed, 258 passed	278 passed
End-to-end	Runner broken	10 specs compile and list; not run without Docker

Commit history

```
$ git log --oneline --reverse
1dea234 chore: baseline import of provided claims application (as received)
fffee08 chore: ignore build outputs and macOS metadata
833c777 docs: add application risk assessment and test strategy
b97ea89 test(claims): repair stale suite so the baseline actually runs
9d0e427 test(claims): exact boundary values and full status transition matrix
1c5d79a test(bff): content-level Kafka consumer tests and logout CSRF contract
40d94cd test(ui): login page and claims table component tests; fix failing shipped tests
f3a6314 test(e2e): Playwright journeys with page objects and the missing global setup
a93ed2a test(load): k6 smoke script for list-claims and cached dashboard
fe44881 docs: AI working journal, verified results and run guide
```

Figure 14.2: Commit history, from the as-received baseline onward.

Next steps, in priority order

1. Start the Docker stack and run the Playwright suite and the existing Testcontainers integration tests, to confirm Bugs 1 and 3 live.
2. Fix Bugs 1–4 using the recommended changes. The failing defect tests turn green and become permanent regression guards.
3. Enforce the Kafka contracts in CI by switching on the schema validator that already exists in the code.
4. Write a concurrency test for Bug 6, then refactor to per-request tokens.
5. Capture a k6 performance baseline and set real thresholds.